

Unit 6

Input/Output Device Management

I/O Management

- One of the important jobs of an Operating System is to manage various I/O devices including mouse, keyboards, touch pad, disk drives, display adapters, USB devices, Bit-mapped screen, LED, Analog-to-digital converter, On/off switch, network connections, audio I/O, printers etc.
- An I/O system is required to take an application I/O request and send it to the physical device, then take whatever response comes back from the device and send it to the application.

Classification of I/O devices

I/O devices can be classified into:

- Block devices or Machine readable devices
- Character devices or User readable devices
- Communication devices

Classification of I/O devices (contd..)

Block devices – A block device is one with which the driver communicates by sending entire blocks of data. For example, Hard disks, USB cameras, Disk-On-Key etc.

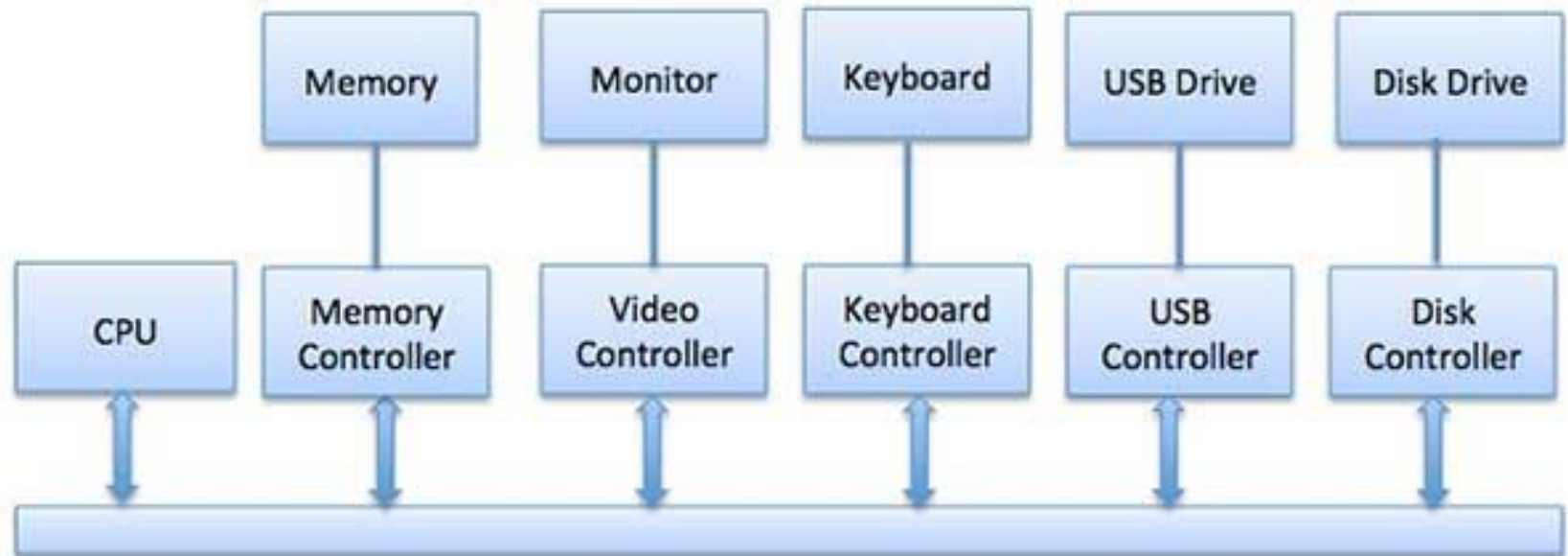
Character devices – A character device is one with which the driver communicates by sending and receiving single characters (bytes, octets). For example, serial ports, parallel ports, sounds cards etc.

Communication devices – It is a hardware capable of transmitting an analog or digital signal over the telephone, other communication wire or wireless. For example: Modem, NIC, Wi-Fi devices.

Device Controllers

- Device drivers are software modules that can be plugged into an OS to handle a particular device. Operating System takes help from device drivers to handle all I/O devices.
- The Device Controller works like an interface between a device and a device driver. I/O units (Keyboard, mouse, printer, etc.) typically consist of a mechanical component and an electronic component where electronic component is called the device controller.
- There is always a device controller and a device driver for each device to communicate with the Operating Systems. A device controller may be able to handle multiple devices. As an interface its main task is to convert serial bit stream to block of bytes, perform error correction as necessary.
- Any device connected to the computer is connected by a plug and socket, and the socket is connected to a device controller. Following is a model for connecting the CPU, memory, controllers, and I/O devices where CPU and device controllers all use a common bus for communication.

Device Controllers (contd..)



Synchronous vs asynchronous I/O

- **Synchronous I/O** – In this scheme CPU execution waits while I/O proceeds
- **Asynchronous I/O** – I/O proceeds concurrently with CPU execution

Communication to I/O Devices

The CPU must have a way to pass information to and from an I/O device. There are three approaches available to communicate with the CPU and Device.

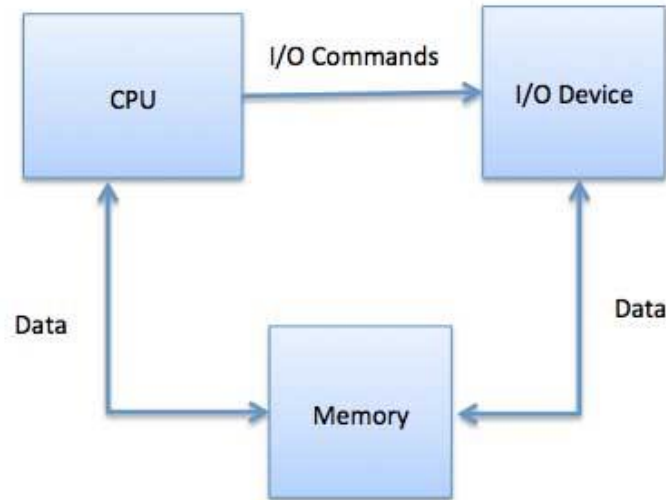
1. Special Instruction I/O
2. Memory-mapped I/O
3. Direct memory access (DMA)

Communication to I/O Devices (contd..)

Special Instruction I/O: This uses CPU instructions that are specifically made for controlling I/O devices. These instructions typically allow data to be sent to an I/O device or read from an I/O device.

Memory-mapped I/O: When using memory-mapped I/O, the same address space is shared by memory and I/O devices. The device is connected directly to certain main memory locations so that I/O device can transfer block of data to/from memory without going through CPU.

Communication to I/O Devices (contd..)



- While using memory mapped IO, OS allocates buffer in memory and informs I/O device to use that buffer to send data to the CPU. I/O device operates asynchronously with CPU, interrupts CPU when finished.
- The advantage to this method is that every instruction which can access memory can be used to manipulate an I/O device. Memory mapped IO is used for most high-speed I/O devices like disks, communication interfaces.

Communication to I/O Devices (contd..)

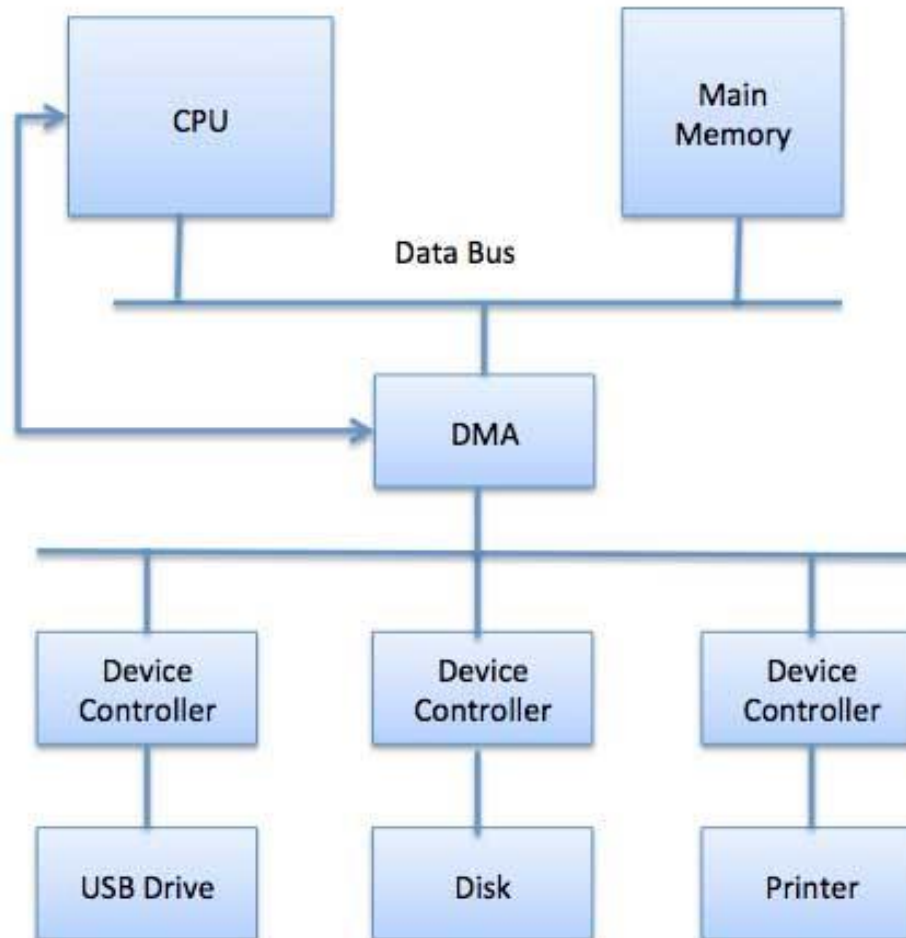
Direct Memory Access (DMA):

Slow devices like keyboards will generate an interrupt to the main CPU after each byte is transferred. If a fast device such as a disk generated an interrupt for each byte, the operating system would spend most of its time handling these interrupts. So a typical computer uses direct memory access (DMA) hardware to reduce this overhead.

Direct Memory Access (DMA) means CPU grants I/O module authority to read from or write to memory without involvement. DMA module itself controls exchange of data between main memory and the I/O device. CPU is only involved at the beginning and end of the transfer and interrupted only after entire block has been transferred.

Direct Memory Access needs a special hardware called DMA controller (DMAC) that manages the data transfers and arbitrates access to the system bus. The controllers are programmed with source and destination pointers (where to read/write the data), counters to track the number of transferred bytes, and settings, which includes I/O and memory types, interrupts and states for the CPU cycles.

DMA



Working procedure of DMA

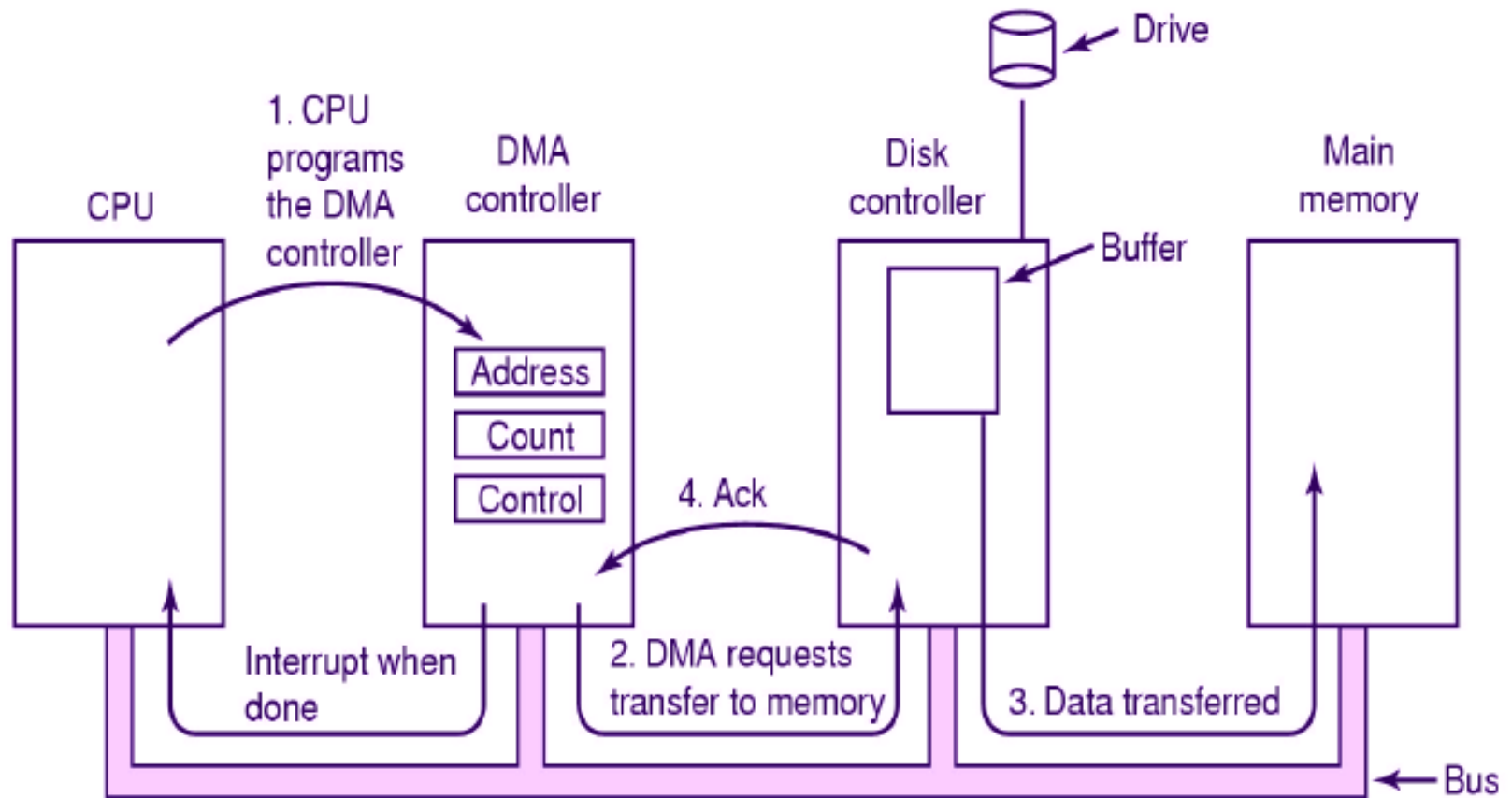


Figure: Operation of a DMA transfer

Working procedure of DMA (contd..)

Step 1: The CPU programs the DMA controller by its registers so it knows what to transfer and where. It also issues a command to disk controller telling it to read data from disk to its internal buffer. When valid data are in disk controller buffer, DMA can begin.

Step 2: The DMA controller initiates the transfer by issuing a read request over the bus to disk controller.

Step 3: Data transferred from disk controller to memory.

Step 4: When transferred completed, the disk controller sends an acknowledgement signal to DMA controller.

Step 5: When transfer completed the DMA controller interrupt the CPU.

Interrupts

- It is the hardware mechanism that enables a device to notify the CPU is called an interrupt. Interrupt forces the CPU to stop what it is doing and start doing something else.

Advantage: Improves efficiency.

Problem: Because of the selfish nature, the first interrupt may not be served if the similar second happened before the time needed to serve the first.

Types of Interrupts

There are three types of interrupts:

- 1. Hardware interrupt:** generated by h/w devices.
- 2. Software interrupt:** generated by programs
- 3. Traps:** generated by CPU itself to indicate that some error or condition occurred for which assistance from the OS is needed.

Interrupt handlers

- An interrupt handler, also known as an interrupt service routine or ISR, is a piece of software or more specifically a callback function in an operating system or more specifically in a device driver, whose execution is triggered by the reception of an interrupt.
- When the interrupt happens, the interrupt procedure does whatever it has to in order to handle the interrupt, updates data structures and wakes up process that was waiting for an interrupt to happen.
- The interrupt mechanism accepts an address – a number that selects a specific interrupt handling routine/function from a small set. In most architectures, this address is an offset stored in a table called the interrupt vector table. This vector contains the memory addresses of specialized interrupt handlers.

Goals or principles of I/O Software

Some of the goals of I/O Software are:

1. Device Independence
2. Uniform naming
3. Error handling
4. Synchronous vs asynchronous transfers
5. Buffering
6. Sharable and dedicated devices

Goals or principles of I/O Software (contd..)

Device Independence: It should be possible to write programs that can access any I/O device without having to specify the device in advance. For example, a program that reads a file as input should be able to read a file on a floppy disk, on a hard disk, or on a CD-ROM, without having to modify the program for each different device.

Uniform Naming: The name of a file or a device should simply be a string or an integer and not depend on the device in any way.

Error Handling: Errors should be handled as close to the hardware as possible. If the controller discovers a read error, it should try to correct the error itself if it can. If it cannot, then the device driver should handle it, perhaps by just trying to read the block again. Many errors are transient, such as read errors caused by specks of dust on the read head, and will go away if the operation is repeated. Only if the lower layers are not able to deal with the problem should the upper layers be told about it. In many cases, error recovery can be done transparently at a low level without the upper levels even knowing about the error.

Goals or principles of I/O Software (contd..)

Synchronous (blocking) vs Asynchronous (interrupt-driven) transfer: Most physical I/O is asynchronous—the CPU starts the transfer and goes off to do something else until the interrupt arrives. User programs are much easier to write if the I/O operations are blocking—after a read system call the program is automatically suspended until the data are available in the buffer. It is up to the operating system to make operations that are actually interrupt-driven look blocking to the user programs.

Buffering: Often data that come off a device cannot be stored directly in its final destination. For example, when a packet comes in off the network, the operating system does not know where to put it until it has stored the packet somewhere and examined it. Also, some devices have severe real-time constraints (for example, digital audio devices), so the data must be put into an output buffer in advance to decouple the rate at which the buffer is filled from the rate at which it is emptied, in order to avoid buffer underruns. Buffering involves considerable copying and often has a major impact on I/O performance.

Sharable and Dedicated devices: Some I/O devices, such as disks, can be used by many users at the same time. Multiple users can open files on the same disk at the same time. Other devices, such as printers, have to be dedicated to a single user until that user is finished and only another user can have the printer.

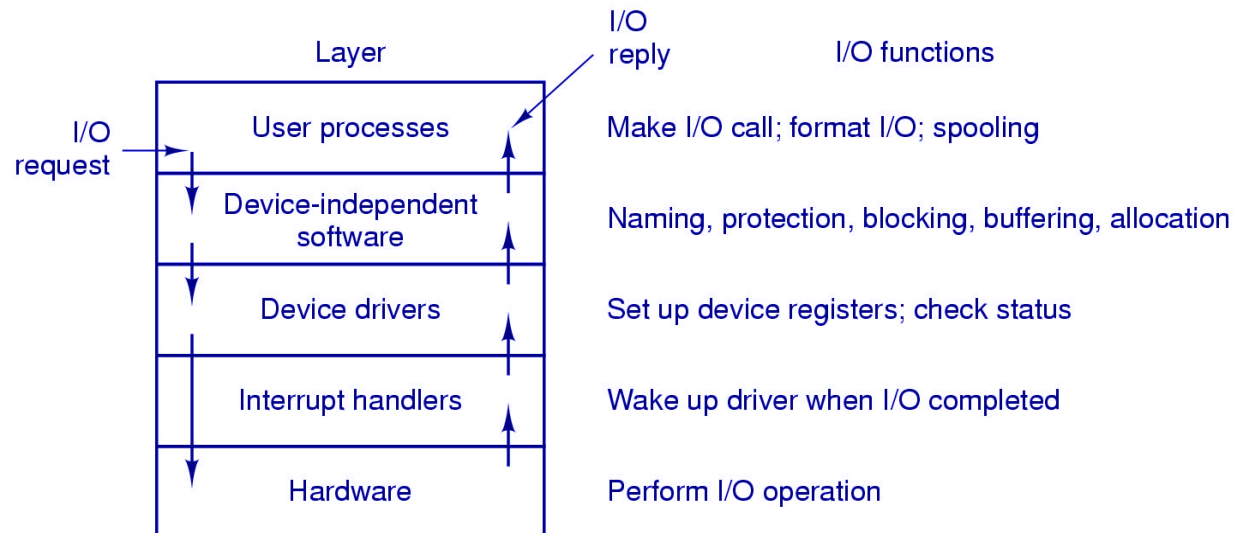
Handling I/O

There are three fundamentally different ways of performing I/O operations:

1. Programmed I/O
2. Interrupt driven I/O
3. I/O using DMA

I/O Software Layers

I/O software is organized into many layers, as shown in figure below. Each layer has a well-defined function to perform and a well-defined interface to the adjacent layers. The functionality and interfaces differ from system to system.



Functions of Device driver

1. Accept read and write requests from the device independent software above it.
2. Initialize the devices, if necessary.
3. Manage power requirement and log events.
4. It checks the status of devices - in use or free.
5. Decides which command to issue to the device controller's device registers if there is command queue.

Device independent I/O Software

The basic function of the device independent-software is to perform the I/O functions that are common to all devices and to provide a uniform interface to the user-level software.

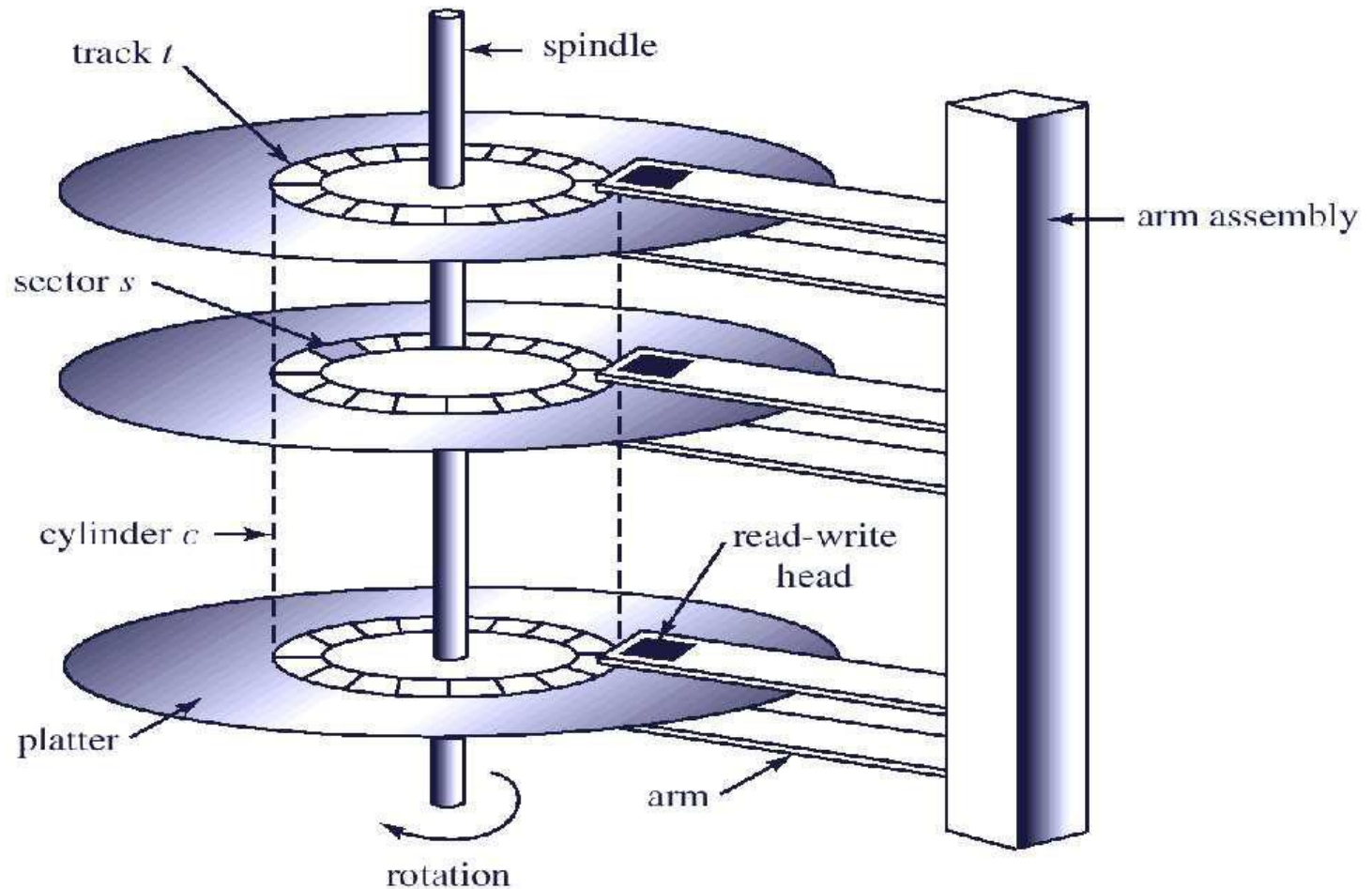
Some common functions in the device-independent software are:

1. Uniform interfacing for device drivers - naming and protection.
2. Buffering.
3. Error reporting.
4. Allocating and releasing dedicated devices.
5. Providing a device-independent block size.

Disk Structure

- Disks like magnetic disk – hard disk, floppy disk are common to all the computers.
- Magnetic disk like hard disk contains multiple disk. Each data storing surface contains separate read/write head. This head is used to read/write on the disk.
- Each disk contains concentric tracks and each track is divided into multiple sectors. Same track on all the surfaces is called a cylinder.

Disk Structure (contd..)



Key terms used in Disk structure

- **Seek Time** – It is the time taken to locate the disk arm to the specified track where the r/w request will be satisfied.
- **Rotational Latency** – It is the time taken by the desired sector of disk to rotate into the position so that r/w head can access it. The algorithm that gives minimum rotational latency is better.
- **Transfer time** – It is the time to transfer the data. It depends on the rotating speed of the disk and number of bytes to be transferred.
- **Disk access Time** = Seek time + Rotational Latency + Transfer time.

- **Disk Response Time:** It is the average time spent by a request waiting to perform its I/O operation.
- **Average response time** is the response time of the all requests.
- **Variance Response Time** is measure of how individual request are serviced with request to average response time. The algorithm that gives minimum variance response time is better.
- **Transmission Time:** To access a particular record, first the arm assembly must be moved to the appropriate cylinder, and then rotate the disk until it is immediately under the r/w head. The time taken to access the whole record is called transmission time.

Numerical 1

Consider a hard disk with 4 surfaces, 64 tracks/surface, 128 sector/track, 256 bytes/sector.

1) What is the capacity of the hard disk?

Disk capacity = $4 * 64 * 128 * 256 = 8,388,608$ bytes = 8 MB

2) The disk is rotating at 3600 RPM, what is the data transfer rate?

60 sec = 3600 rotations, so 1 sec = 60 rotations

Data transfer rate = no of rotations/sec * track capacity * no of surfaces

So, Data transfer rate = $60 * 128 * 256 * 4 = 7,864,320$ Bytes/Sec i.e. 7.5 MB/Sec

Numerical 1 (Contd ...)

3) The disk is rotating at 3600 RPM, What is the average access time?

Since, we don't have seek time, amount of data to be transferred, we consider them as 0.

So, Average access time = Average rotational latency

Here, 3600 rotations in 60 sec, 60 rotations in 1 sec

So, Rotational Latency = $(1/60)$ sec = 16.67 msec

Average rotational latency = $(16.67/2)$ msec = 8.33 msec

Average access time = 8.33 msec

Disk Scheduling

- Disk scheduling is a technique used by the operating system to schedule multiple requests for accessing the disk.
- The main purpose of disk scheduling algorithm is to select a disk request from the queue of IO requests and decide the schedule when this request will be processed.
- Goal of Disk Scheduling Algorithm:
 - ✓ Fairness
 - ✓ High throughput
 - ✓ Minimal traveling head time

Disk Scheduling Algorithms

1. FCFS (First-Come, First Served) Scheduling
2. SSTF (Shortest Seek Time First) Scheduling
3. SCAN Scheduling (Elevator Algorithm)
4. C-SCAN Scheduling (Circular SCAN algorithm)
5. LOOK Scheduling
6. C-LOOK Scheduling

FCFS Scheduling

- First Come First Served
- It is the simplest among all.
- The requests are addressed in the order they arrive in the disk queue
- Features of FCFS:
 - Perform operations in order requested
 - No reordering of work queue
 - No starvation: every request is serviced
 - Poor performance

FCFS Scheduling (contd..)

- Advantages

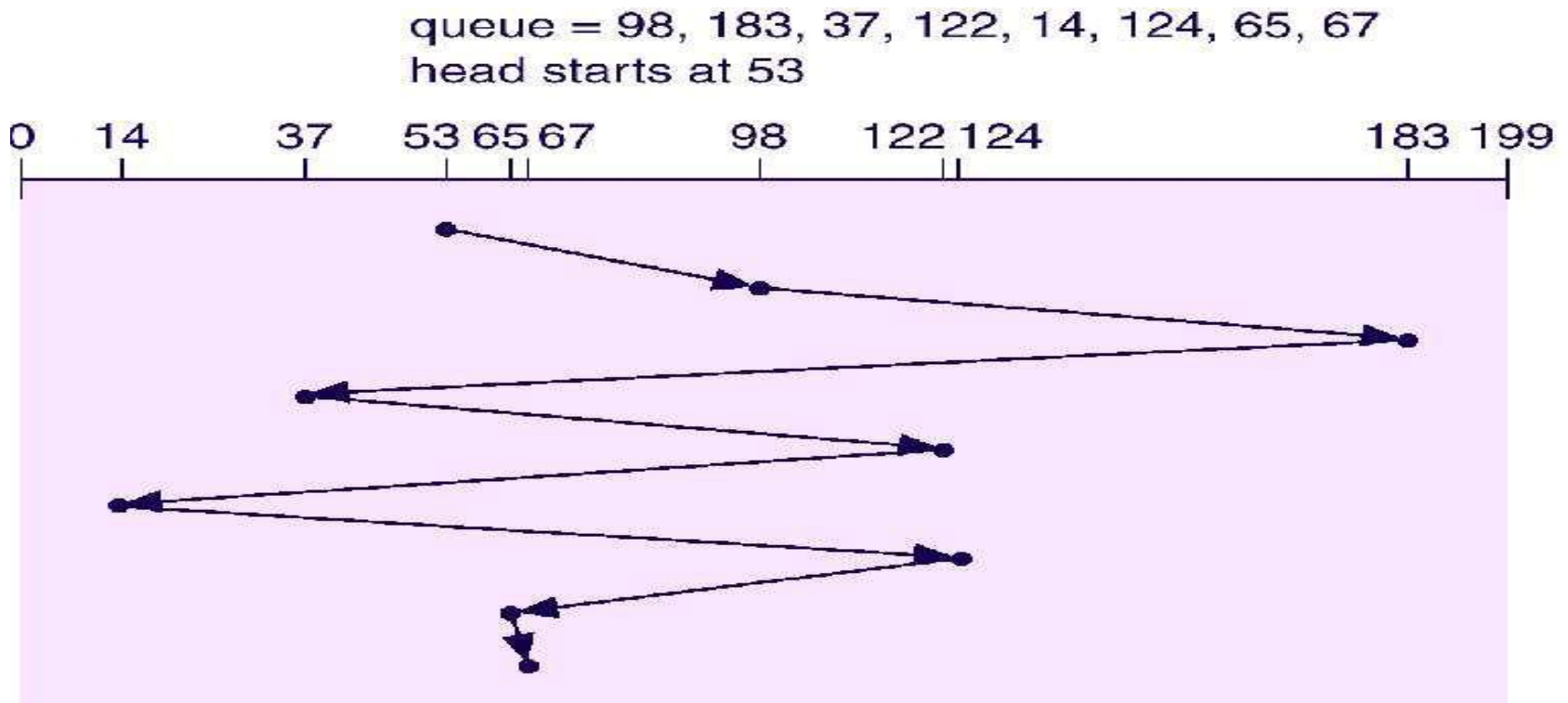
- Every requests gets a fair chance
- No indefinite postponement

- Disadvantages

- Doesn't try to optimize seek time
- May not provide the best possible service

FCFS Scheduling Example 1

Given the following queue {98, 183, 37, 122, 14, 124, 65, 67} with the r/w head initially at track 53 and the tail track being at 199. Find total head movement by using FCFS disk scheduling algorithm.



Total head movement = $((98-53)+(183-98)+(183-37)+(122-37)+(122-14)+(124-14)+(124-65)+(67-65)) = 640$ cylinders.

Example 2

- Suppose that a disk has 100 cylinders, numbered 0 to 99. The drive is currently serving a request at cylinder 43 and previous request was at cylinder 25. The queue of pending request, in FIFO order is 86, 70, 13, 74, 48, 9, 22, 50, 30. Starting from the current head position, what is total distance (in cylinders) that the disk arm moves to satisfy all pending request for FCFS Scheduling algorithm.

SSTF Scheduling

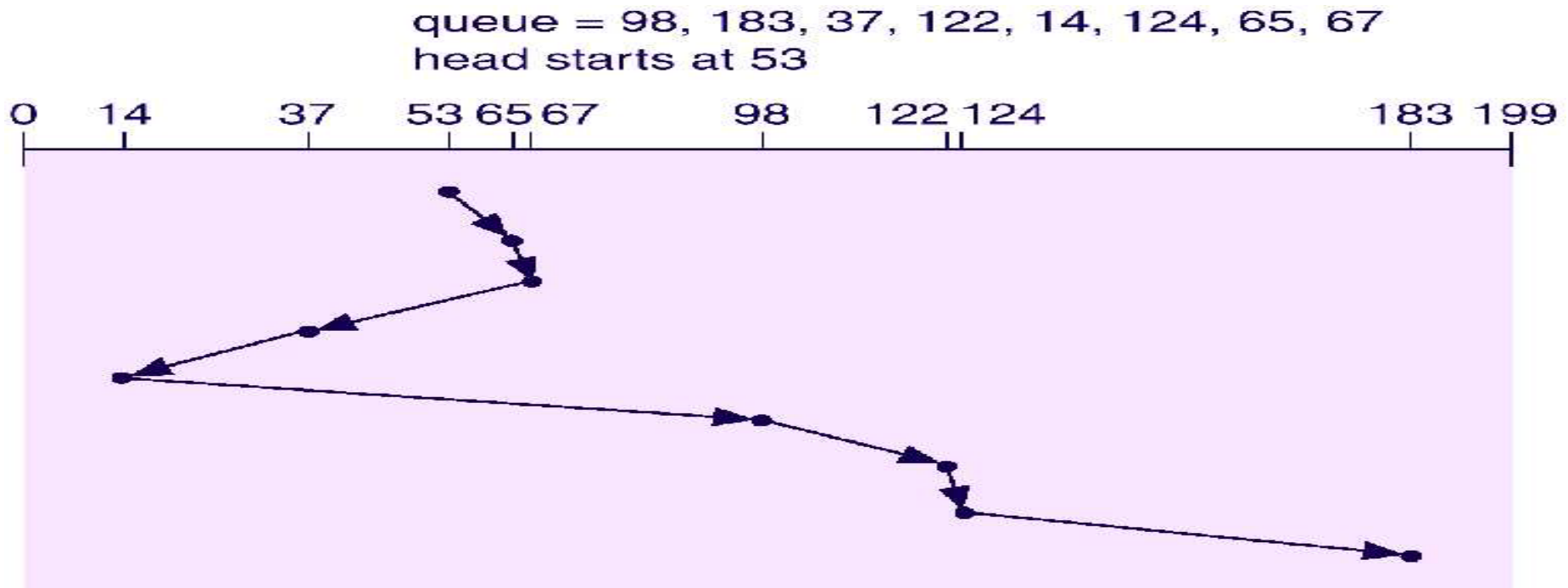
- Shortest Seek Time First.
- The requests having shortest seek time are executed first.
- Seek time of every request is calculated in advance in queue and then scheduled accordingly.
- The request near the disk arm will be executed first.

SSTF Scheduling (contd..)

- Advantages
 - Average Response Time decreases
 - Throughput increases
- Disadvantages
 - Overhead to calculate seek time in advance
 - Can cause starvation for a request if it has higher seek time as compared to incoming requests
 - High variance of response time as SSTF favors only some requests.

SSTF Scheduling Example 1

Given the following queue {98, 183, 37, 122, 14, 124, 65, 67} with the r/w head initially at track 53 and the tail track being at 199. Find total head movement by using SSTF disk scheduling algorithm.



Total head movement = $((65-53)+(67-65)+(67-37)+(37-14)+(98-14)+(122-98)+(124-122)+(183-124)) = 12+2+30+23+84+24+2+59 = 236$ cylinders.

Example 2

- Suppose that a disk has 100 cylinders, numbered 0 to 99. The drive is currently serving a request at cylinder 43 and previous request was at cylinder 25. The queue of pending request, in FIFO order is 86, 70, 13, 74, 48, 9, 22, 50, 30. Starting from the current head position, what is total distance (in cylinders) that the disk arm moves to satisfy all pending request for SSTF Scheduling algorithm.

SCAN Scheduling

- In SCAN scheduling, the disk arm moves into particular direction and services the requests coming in its path and after reaching the end of the disk, it reverses its direction and again services the request arriving in its path.
- It is also called as elevator algorithm as it works like an elevator.

SCAN Scheduling (Contd..)

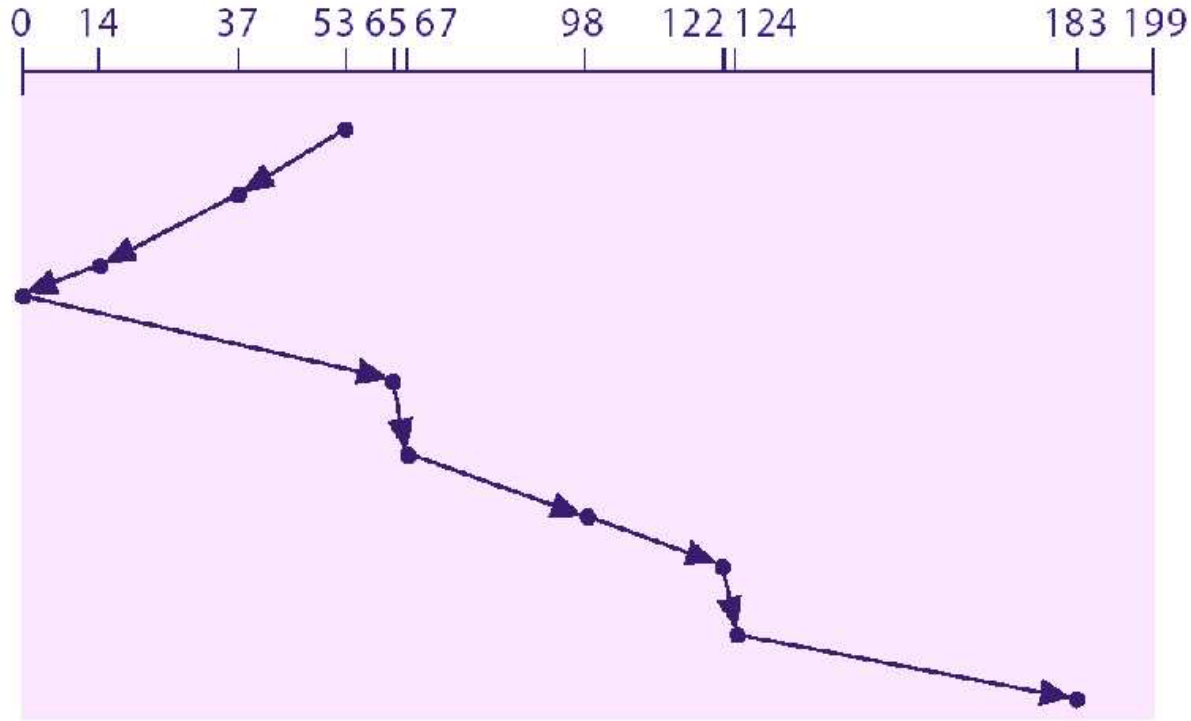
- Advantages
 - High throughput
 - Low variance of response time
 - Average response time
- Disadvantages
 - Long waiting time for request for locations just visited by disk arm.

SCAN Scheduling Example 1

Given the following queue {98, 183, 37, 122, 14, 124, 65, 67} with the r/w head initially at track 53 and the tail track being at 199. Find total head movement by using SCAN disk scheduling algorithm.

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



Total head movement = $((53-37)+(37-14)+(14-0)+(65-0)+(67-65)+(98-67)+(122-98)+(124-122)+(183-124)) = 16+23+14+65+2+31+24+2+59 = 236$ cylinders.

Example 2

- Suppose that a disk has 100 cylinders, numbered 0 to 99. The drive is currently serving a request at cylinder 43 and previous request was at cylinder 25. The queue of pending request, in FIFO order is 86, 70, 13, 74, 48, 9, 22, 50, 30. Starting from the current head position, what is total distance (in cylinders) that the disk arm moves to satisfy all pending request for SCAN Scheduling algorithm.

C-SCAN Scheduling

- Circular SCAN Scheduling.
- It is the modification of SCAN scheduling, in which the disk arm instead of reversing its direction goes to the other end of the disk and starts servicing the requests from there. So, the disk arm moves in a circular fashion.

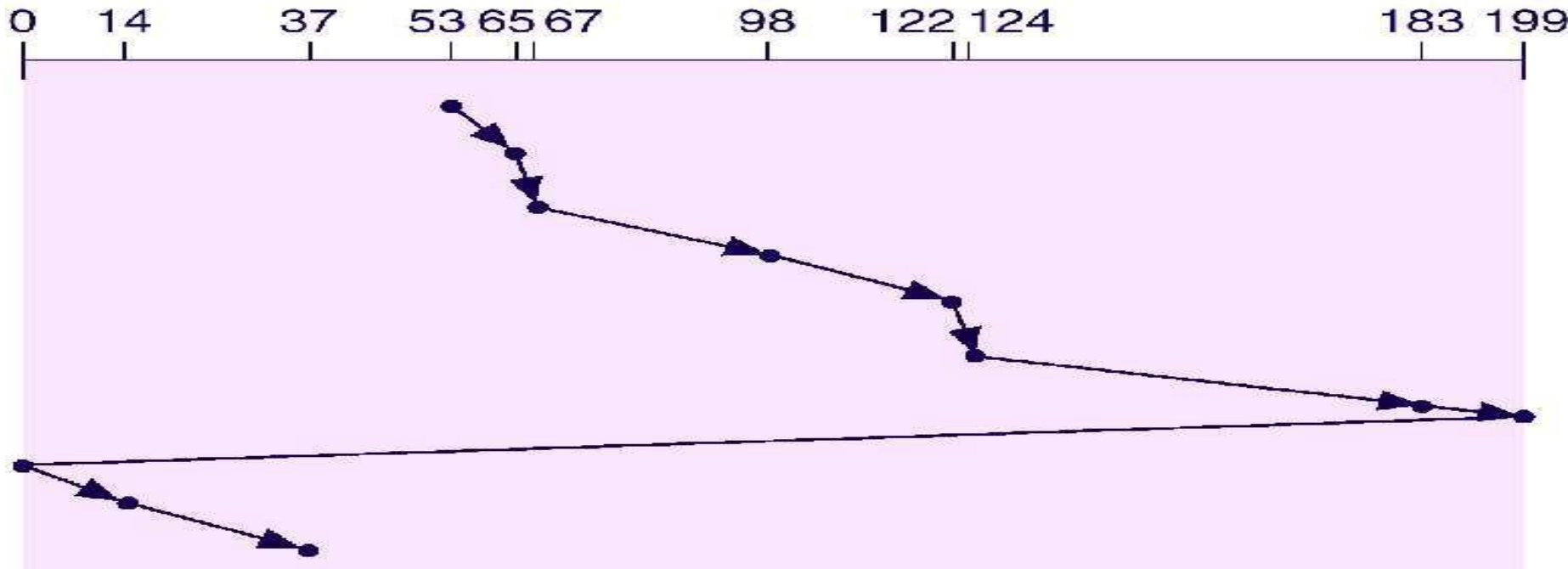
C-SCAN Scheduling (Contd..)

- Advantages
 - The waiting time for the cylinders just visited by the head is reduced as compared to the SCAN algorithm.
 - It provides uniform waiting time.
 - It provides better response time
- Disadvantages
 - It causes more seek movements as compared to SCAN algorithms.
 - It causes the head to move till the end of the disk even if there are no requests to be serviced.

C-SCAN Scheduling Example 1

Given the following queue {98, 183, 37, 122, 14, 124, 65, 67} with the r/w head initially at track 53 and the tail track being at 199. Find total head movement by using C-SCAN disk scheduling algorithm.

queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53



Total head movement = $((65-53)+(67-65)+(98-67)+(122-98)+(124-122)+(183-124)+(199-183)+(14-0)+(37-14)) = 12+2+31+24+2+59+16+14+23 = 183$ cylinders.

Example 2

- Suppose that a disk has 100 cylinders, numbered 0 to 99. The drive is currently serving a request at cylinder 43 and previous request was at cylinder 25. The queue of pending request, in FIFO order is 86, 70, 13, 74, 48, 9, 22, 50, 30. Starting from the current head position, what is total distance (in cylinders) that the disk arm moves to satisfy all pending request for C-SCAN Scheduling algorithm.

LOOK Scheduling Algorithm

- It is similar to SCAN disk scheduling algorithm except the difference that the disk arm in spite of going to the end of the disk goes only to the last request to be serviced in front of the head and then reverses its direction from there only.
- The main difference between SCAN and LOOK is:
SCAN scans all the cylinders of the disk starting from one end to the other end even if there are no requests at the end whereas LOOK scans all the cylinders of the disk starting from the first request at one end to the last request at the other end.

LOOK Scheduling Algorithm (contd..)

- Advantages:
 - It does not cause the head to move till the end of the disk when there are no requests to be serviced.
 - It provides better performance as compared to SCAN disk scheduling algorithm.
 - It doesn't lead to starvation.
 - It provides low variance in response time and waiting time.
- Disadvantages:
 - There is no overhead of finding the end requests.
 - It causes long waiting time for the cylinders just visited by the head.

LOOK Scheduling Example 1

Given the following queue {98, 183, 37, 122, 14, 124, 65, 67} with the r/w head initially at track 53 and the tail track being at 199. Find total head movement by using C-SCAN disk scheduling algorithm.

Total head movement = $((65-53)+(67-65)+(98-67)+(122-98)+(124-122)+(183-124)+(183-37)+(37-14)) = 12+2+31+24+2+59+146+23 = 299$ cylinders.

Example 2

- Suppose that a disk has 100 cylinders, numbered 0 to 99. The drive is currently serving a request at cylinder 43 and previous request was at cylinder 25. The queue of pending request, in FIFO order is 86, 70, 13, 74, 48, 9, 22, 50, 30. Starting from the current head position, what is total distance (in cylinders) that the disk arm moves to satisfy all pending request for LOOK Scheduling algorithm.

C-LOOK Scheduling Algorithm

- It is an improved version of the LOOK algorithm.
- Head starts from the first request at one end of the disk and moves towards the last request at the other end servicing all the requests in between.
- After reaching the last request at the other end, head reverses its direction. It then returns to the first request at the starting end without servicing any request in between.

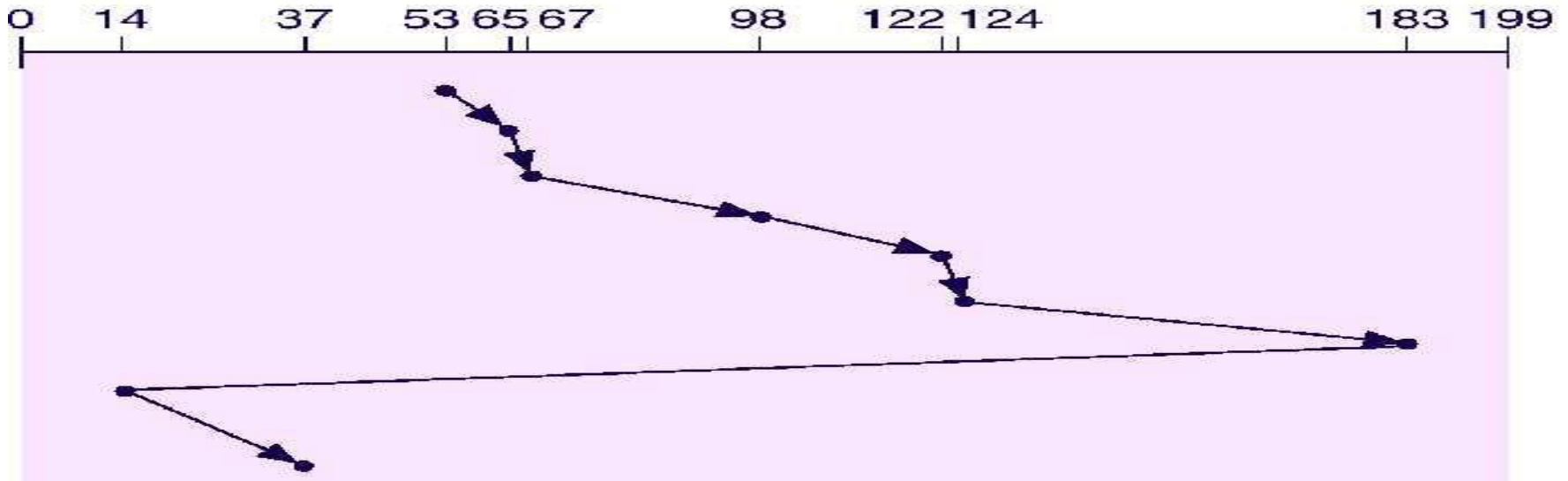
C-LOOK Scheduling Algorithm

- Advantages
 - It doesn't cause the head to move till the end of the disk when there are no requests to be serviced.
 - It reduces the waiting time for the cylinders just visited by the head.
 - It provides better performance as compared to LOOK algorithm.
 - It doesn't lead to starvation.
 - It provides low variance in response time and waiting time.
- Disadvantages
 - It contains overhead of finding the end requests.

C-LOOK Scheduling Example 1

Given the following queue {98, 183, 37, 122, 14, 124, 65, 67} with the r/w head initially at track 53 and the tail track being at 199. Find total head movement by using C-SCAN disk scheduling algorithm.

queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53



Total head movement = $((65-53)+(67-65)+(98-67)+(122-98)+(124-122)+(183-124)+(183-14)+(37-14))$
 $= 12+2+31+24+2+59+169+23 = 322$ cylinders.

Example 2

- Suppose that a disk has 100 cylinders, numbered 0 to 99. The drive is currently serving a request at cylinder 43 and previous request was at cylinder 25. The queue of pending request, in FIFO order is 86, 70, 13, 74, 48, 9, 22, 50, 30. Starting from the current head position, what is total distance (in cylinders) that the disk arm moves to satisfy all pending request for C-LOOK Scheduling algorithm.